

PÊNDULO INVERTIDO ROTACIONAL DE FURUTA

Documentação técnica do desenvolvimento, identificação e controle

Projeto FURUTA V0

Snapshot documental: FURUTA_V0.zip
09 de agosto de 2026

Documento de consolidação técnica

Controle e base documental

Este documento consolida o desenvolvimento do projeto FURUTA V0 a partir do código-fonte fornecido no arquivo FURUTA_V0.zip, do histórico preservado nos diretórios de fases e dos registros experimentais apresentados durante o desenvolvimento. A implementação corrente é tratada como fonte prioritária para parâmetros de operação, enquanto arquivos de fases anteriores são usados como registro histórico da evolução do projeto.

Elemento	Fonte primária usada neste documento
Configuração atual	platformio.ini e lib/FurutaSystem/src/FurutaConfig.h
Implementação corrente	src/fase_10_integracao_final/main.cpp e lib/FurutaSystem/src/*
Sensor e estimativa de velocidade	lib/PendulumPosition/src/* e lib/PendulumVelocity/src/*
Atuador	lib/MotorPosition/src/* e lib/MotoreVelocity/src/*
Identificação	src/fase_07_identifica/main.cpp
Controle PD	src/fase_08_projetoPD/main.cpp
Controle de quatro estados	src/fase_09_projetoMotorPendulo/main.cpp e FourStateController
Resultados experimentais	registros seriais fornecidos durante os ensaios

Regra de rastreabilidade: Quando existe divergência entre um README histórico e o código corrente, o código corrente é usado para descrever o estado atual. Divergências relevantes são registradas explicitamente no Apêndice E; elas não são silenciosamente reconciliadas.

Escopo atual: O projeto consolidado estabiliza o pêndulo a partir de uma captura manual próxima à vertical. O swing-up autônomo não está implementado no snapshot atual e permanece como etapa futura.

Resumo executivo

O projeto implementa um pêndulo invertido rotacional de Furuta com Arduino Nano, motor de passo NEMA 17 acionado por A4988 e sensor magnético absoluto AS5600 no pêndulo. O desenvolvimento foi deliberadamente incremental: primeiro foram validados ambiente, sensor e motor; em seguida foram estabelecidas referências e limites, caracterizado o atuador, refinada a estimativa de velocidade angular, identificado um modelo local próximo à vertical, implementado um controlador PD e, finalmente, um controlador por realimentação de quatro estados.

A versão consolidada utiliza período de controle nominal de 4 ms (250 Hz), telemetria de 25 Hz, velocidade máxima comandada do braço de 180 graus/s e saturação da aceleração de controle em ± 600 graus/s². O vetor de estado usado no controlador é $x = [\phi, \dot{\phi}, \beta, \dot{\beta}]^T$. A posição do pêndulo β é medida pelo AS5600; $\dot{\beta}$ é estimada por regressão linear em uma janela de sete amostras; ϕ é inferida pela contagem de passos emitidos; e $\dot{\phi}$ corresponde à referência de velocidade comandada ao motor.

O modelo local consolidado é $\ddot{\beta} = 21.7 \beta - 3.28 \dot{\beta} - 0.75 u$, em unidades SI. O controlador final foi projetado por posicionamento conjunto de polos em -0.7, -1.0, -4.0 e -6.0, resultando na lei $u = 0.77419355 \phi$

+ 2.31978594 phiDot + 95.71078856 beta + 14.31971458 betaDot. Um ensaio registrado de 100 s na Fase 09 terminou por limite de tempo, com máximo absoluto de beta igual a 1.758 graus, máximo absoluto de phi igual a 31.950 graus, aceleração máxima de 370.7 graus/s² e nenhuma saturação. Outro ensaio terminou por BETA_LIMIT após 25.541 s, mostrando que a estabilização é funcional, mas a robustez ainda deve ser caracterizada sistematicamente.

Sumário

1. Introdução
 2. Sistema físico e arquitetura experimental
 3. Convenções, referências e unidades
 4. Ambiente de desenvolvimento e validação da plataforma
 5. Bibliotecas e arquitetura de software
 6. Aquisição da posição do pêndulo
 7. Estimativa da velocidade angular do pêndulo
 8. Caracterização e validação do atuador
 9. Metodologia de identificação experimental
 10. Modelo local identificado
 11. Primeiro controlador local: PD do pêndulo
 12. Captura e entrada no modo BALANCE
 13. Da correção independente do braço ao controle conjunto
 14. Modelo aumentado de quatro estados
 15. Projeto do controlador de quatro estados
 16. Validação experimental do BALANCE
 17. Refatoração e integração final - Fase 10
 18. Telemetria do vetor de estado e do controle
 19. Procedimento operacional atual
 20. Resultados consolidados, limitações e riscos técnicos
 21. Próximas etapas
 22. Conclusão
- A. Apêndice A - Pinagem e parâmetros principais**
- B. Apêndice B - Mapa das fases**
- C. Apêndice C - Comandos seriais da implementação atual**
- D. Apêndice D - Estrutura de diretórios**
- E. Apêndice E - Auditoria de consistência do repositório**

PARTE I

Fundamentos do projeto

Descrição do sistema, escopo e convenções adotadas

1. Introdução

1.1 Objetivo

O objetivo técnico do projeto é desenvolver uma plataforma experimental capaz de manter um pêndulo rotacional de Furuta próximo à posição vertical superior, que é um equilíbrio naturalmente instável. O desenvolvimento também busca produzir uma arquitetura de software modular e uma metodologia experimental suficientemente clara para permitir evolução posterior para swing-up, estimação de estados e estratégias de controle mais robustas.

1.2 Estratégia incremental

Em vez de implementar diretamente um controlador final, o projeto foi dividido em fases. Cada fase isola uma pergunta experimental: o ambiente compila e grava? o sensor mede corretamente? a velocidade derivada é utilizável? o motor respeita posição e limites? qual aceleração pode ser comandada com margem? qual modelo local descreve a dinâmica? um PD estabiliza o pêndulo? por que o braço deriva? a realimentação dos quatro estados resolve o problema?

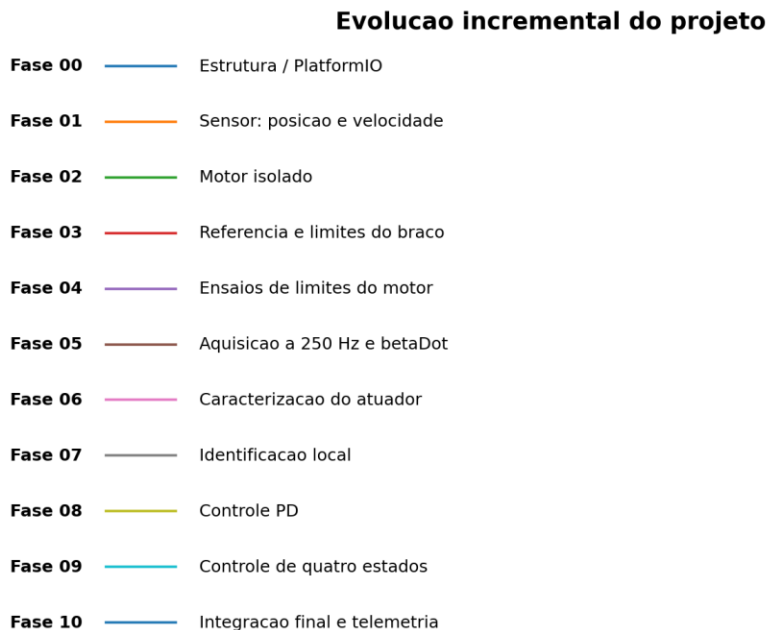


Figura 1 - Evolução incremental do desenvolvimento preservada no repositório.

Essa organização permitiu que as hipóteses fossem validadas progressivamente. O projeto atual chegou à Fase 10, na qual o código de alto nível foi reduzido a uma interface mínima e a lógica de aquisição, controle, segurança e telemetria foi encapsulada em bibliotecas próprias.

1.3 Estado atual

- Aquisição angular do pêndulo validada com AS5600.
- Estimativa de betaDot implementada por regressão linear em sete amostras.
- Controle do motor em posição e posteriormente em aceleração comandada.
- Referência do braço baseada em zero manual e contagem de passos.
- Modelo local próximo à vertical identificado experimentalmente.
- Controle PD capaz de estabilizar beta localmente, porém sem conter a posição do braço.
- Controle de quatro estados capaz de estabilizar o pêndulo e manter o braço dentro da região operacional em ensaios representativos.

- Integração final com telemetria CSV de $x(t)$, $u_{Raw}(t)$ e $u(t)$.

2. Sistema físico e arquitetura experimental

2.1 Hardware principal

Componente	Descrição consolidada
Controlador	Arduino Nano / ATmega328P
Atuador	Motor de passo NEMA 17 17HS8401
Driver	A4988
Alimentação registrada	12 V; fonte de 2 A máx.
Sensor do pêndulo	AS5600, encoder magnético absoluto de 12 bits
Microstepping	1/8
Passos inteiros do motor	200 passos/revolução

Com 200 passos inteiros por revolução e microstepping 1/8, o comando nominal corresponde a 1600 micropassos por volta, ou aproximadamente 4.444 micropassos por grau. A resolução angular nominal de um micropasso do braço é, portanto, 0.225 grau. Essa é uma resolução de comando; não deve ser interpretada como precisão mecânica absoluta.

2.2 Geometria registrada

Parâmetro	Valor registrado
Comprimento do pêndulo	30 cm
Massa na extremidade do pêndulo	71,45 g
Centro de massa informado	18,473 cm a partir do eixo
Comprimento do braço rotacional	20 cm
Massa do braço + conjunto do sensor	261,55 g

Os valores geométricos acima são aqueles registrados no README do projeto. Eles não são usados diretamente pelo controlador atual, porque a estratégia adotada passou pela identificação experimental do modelo local em torno da vertical.

2.3 Variáveis do sistema

$$x = [\phi, \dot{\phi}, \beta, \dot{\beta}]^T$$

Símbolo	Significado	Origem no firmware
ϕ	posição angular do braço	contagem de passos / currentPosition do AccelStepper

Símbolo	Significado	Origem no firmware
phiDot	velocidade do braço usada no estado	referência de velocidade comandada pelo MotorVelocity
beta	ângulo do pêndulo em torno da vertical superior	AS5600 + referência DOWN + deslocamento de pi
betaDot	velocidade angular estimada do pêndulo	regressão linear em janela móvel
u	aceleração angular comandada ao braço	saída do controlador, limitada e integrada pelo MotorVelocity

Interpretação fundamental: phi e phiDot não são medidos por um encoder do braço. São estados inferidos a partir do comando de passos e da referência de velocidade. Essa diferença é central para compreender as limitações do modelo e do controlador.

3. Convenções, referências e unidades

3.1 Referência do braço

O braço é referenciado manualmente no centro mecânico. Com o motor desabilitado, o operador posiciona fisicamente o braço na referência central e envia o comando Z. A posição passa então a ser tratada como $\phi = 0$. Ao desabilitar o driver, o software considera a referência perdida, porque o braço pode ser deslocado manualmente sem realimentação de posição.

Historicamente foram usados limites físicos aproximados de -90 a +90 graus. A implementação consolidada adota limites absolutos de software de -80 a +80 graus e, durante BALANCE, encerra o controle em +/-75 graus para preservar uma margem adicional de 5 graus.

3.2 Referência do pêndulo

Nas fases iniciais, a referência $\beta = 0$ chegou a ser definida diretamente na posição de interesse. A partir da Fase 08, a calibração foi tornada mais reprodutível: o comando T é executado com o pêndulo livre, parado e para baixo. Essa posição é registrada como DOWN; a vertical superior é então calculada como DOWN + 180 graus.

$$\beta = 0 \rightarrow \text{vertical superior} \quad / \quad \beta = \pm\pi \rightarrow \text{vertical inferior}$$

Na Fase 10, a função de biblioteca ainda se chama `calibrateTop()`, por herança da evolução do código, mas ela é deliberadamente chamada com o pêndulo para baixo. O método `betaFromDownReference()` subtrai π e aplica wrap para o intervalo angular local em torno do topo.

3.3 Unidades

A interface humana e vários testes históricos usam graus, graus/s e graus/s². O controlador final e o vetor de estado usam unidades SI: rad, rad/s e rad/s². O MotorVelocity converte a aceleração aplicada de volta para graus/s² antes de integrar a referência de velocidade e comandar o AccelStepper.

PARTE II

Validação da plataforma e software

Ambiente, dependências e arquitetura modular

4. Ambiente de desenvolvimento e validação da plataforma

4.1 VS Code e PlatformIO

O projeto é organizado para VS Code + PlatformIO. O arquivo platformio.ini define atmelavr, placa nanoatmega328, framework Arduino e protocolo de upload arduino. O ambiente padrão atual é fase_10_integracao_final. O monitor serial compartilhado está configurado em 250000 baud com filtros time e colorize.

```
[platformio]
default_envs = fase_10_integracao_final

[env]
platform = atmelavr
board = nanoatmega328
framework = arduino
upload_protocol = arduino
monitor_speed = 250000
```

4.2 Organização por ambientes

Cada diretório em src/fase_* corresponde a uma etapa experimental. O build_src_filter seleciona apenas o main.cpp da fase escolhida, evitando que múltiplos setup()/loop() coexistam no mesmo build. Essa organização foi útil durante o desenvolvimento incremental e preserva o histórico conceitual do projeto.

4.3 Evolução da comunicação serial

Os programas das fases iniciais usam 115200 baud. A partir da identificação e dos controladores mais recentes, foi adotado 250000 baud. O platformio.ini atual aplica 250000 globalmente; por isso, os ambientes históricos antigos devem ser tratados como snapshots de desenvolvimento e não como builds de regressão garantidos no estado atual do repositório.

4.4 Upload e bootloader

O README registra que o Arduino Nano utilizado exigiu tratamento de bootloader antigo durante a validação inicial. O platformio.ini corrente não contém uma opção explícita adicional de bootloader; portanto, qualquer ajuste específico de upload depende da configuração local que foi validada durante o desenvolvimento.

5. Bibliotecas e arquitetura de software

5.1 Dependências externas

Biblioteca	Versão encontrada no cache .pio	Responsabilidade
AS5600 - Rob Tillaart	0.6.7	leitura do encoder magnético AS5600
AccelStepper	1.64	geração de passos, posição e velocidade do NEMA 17

O platformio.ini declara as dependências por nome, sem fixar versão. As versões acima são as resolvidas no cache .pio presente no snapshot analisado. Para reprodutibilidade de longo prazo, recomenda-se futuramente fixar as versões no arquivo de configuração.

5.2 Bibliotecas próprias

Módulo	Responsabilidade
PendulumPosition	leitura angular, referência circular e beta
PendulumVelocity	unwrap e estimativa de betaDot por regressão linear
MotorPosition	movimentos angulares posicionais usados nas fases iniciais
MotorVelocity	integração de aceleração para referência de velocidade, saturação e limites
StateVector	estrutura $x = [\text{phi}, \text{phiDot}, \text{beta}, \text{betaDot}]$
FourStateController	lei $u = Kx$, componentes e saturação de u
BalanceTelemetry	telemetria CSV desacoplada do controle
FurutaSystem	orquestração da Fase 10: estados, comandos, aquisição, controle e segurança

Arquitetura funcional consolidada - Fase 10

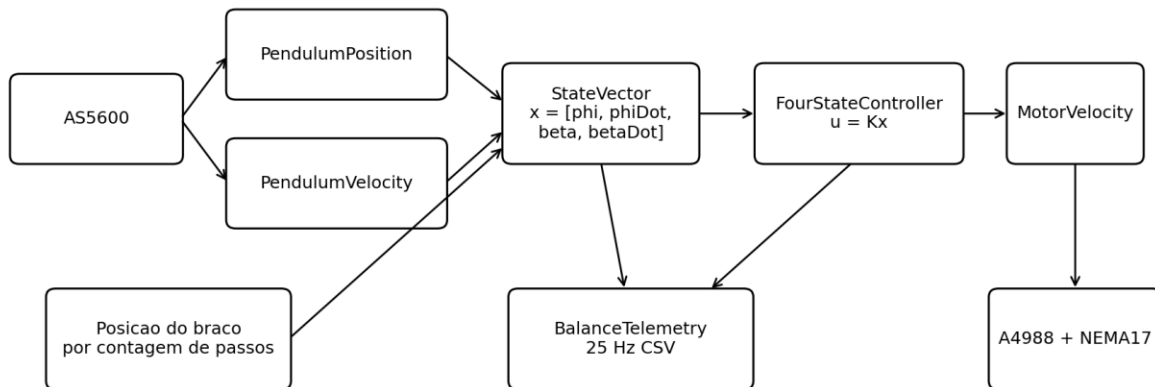


Figura 2 - Arquitetura funcional consolidada na Fase 10.

5.3 Main mínimo da Fase 10

A refatoração final deslocou a complexidade de coordenação para FurutaSystem. O arquivo de aplicação passou a ter apenas a inicialização e a chamada periódica de `update()`:

```
#include <Arduino.h>
#include <FurutaSystem.h>
FurutaSystem furuta;
void setup()
{
    furuta.begin();
}
void loop()
{
    furuta.update();
}
```

Essa forma não reduz a complexidade total do sistema, mas melhora a separação de responsabilidades: parâmetros ficam em `FurutaConfig.h`, controle em `FourStateController`, telemetria em `BalanceTelemetry` e a lógica operacional em `FurutaSystem`.

PARTE III

Caracterização dos componentes

Medição angular, velocidade estimada e atuador

6. Aquisição da posição do pêndulo

6.1 AS5600 e resolução

O AS5600 fornece uma leitura angular absoluta de 12 bits, portanto 4096 posições por revolução. Uma contagem representa aproximadamente 0.087890625 grau ou 0.00153398 rad. A classe PendulumPosition lê o ângulo bruto, armazena o ângulo absoluto e calcula uma diferença circular em relação à referência.

$$1 \text{ count} = 2 \cdot \pi / 4096 = 0.00153398 \text{ rad} = 0.087890625 \text{ deg}$$

6.2 Diferença circular

Para evitar descontinuidades artificiais na passagem 0/4095, a diferença entre a leitura atual e a referência é limitada ao intervalo -2048 ... +2047 contagens. Isso permite que beta permaneça localmente contínuo ao redor da referência escolhida.

6.3 Calibração por média circular local

A função calibrateTop() realiza várias leituras, toma uma primeira leitura como referência e acumula diferenças circulares das amostras subsequentes. Na configuração corrente são utilizadas 32 amostras com intervalo de 2 ms. A média das diferenças é somada à referência inicial e normalizada para 0 ... 4095.

6.4 Fase 05 - aquisição determinística

A Fase 05 consolida a aquisição em período nominal de 4000 us, ou 250 Hz. A telemetria dessa fase é decimada por 2, portanto transmitida a 125 Hz. O código contabiliza amostras, amostras perdidas e erros I²C, permitindo separar problemas do sensor de problemas de temporização.

7. Estimativa da velocidade angular do pêndulo

7.1 Por que a derivada de duas amostras é inadequada

Com $T_s = 4 \text{ ms}$, uma variação de apenas uma contagem do AS5600 entre duas amostras produziria uma derivada de aproximadamente 0.3835 rad/s. Assim, mesmo um pêndulo quase parado pode apresentar impulsos artificiais se betaDot for calculada simplesmente como diferença de duas amostras.

$$0.00153398 \text{ rad} / 0.004 \text{ s} = 0.3835 \text{ rad/s por count}$$

7.2 Janela de sete amostras

A solução consolidada usa PendulumVelocity::WINDOW_SIZE = 7. Primeiro o ângulo circular é desenrolado (unwrap) para uma variável contínua. Em seguida é ajustada uma reta beta(t) às amostras por mínimos quadrados; a inclinação é tomada como betaDot. A janela cobre nominalmente seis intervalos de 4 ms, isto é, 24 ms.

$$\text{betaDot} = \text{sum}[(t_i - t_{\text{mean}})(\text{beta}_i - \text{beta}_{\text{mean}})] / \text{sum}[(t_i - t_{\text{mean}})^2]$$

O primeiro tempo da janela é subtraído dos demais antes do cálculo. Essa decisão evita usar diretamente valores grandes de micros() em float no ATmega328P e melhora a condição numérica do estimador. A velocidade só é marcada como pronta depois que as sete amostras válidas estão disponíveis.

7.3 Consequência dinâmica do estimador

A regressão reduz fortemente o efeito da quantização, mas introduz memória. Para uma janela simétrica de 24 ms, a informação de velocidade representa aproximadamente a tendência no centro da janela, o que corresponde a

um atraso efetivo da ordem de 12 ms. Esse valor é uma interpretação de engenharia do estimador e não um atraso implementado explicitamente no código.

Decisão consolidada: A robustez da estimativa de betaDot foi considerada mais importante do que a resposta instantânea ruidosa da derivada de duas amostras. O controlador final foi validado experimentalmente usando esse estimador.

8. Caracterização e validação do atuador

8.1 Controle posicional inicial

As Fases 02 a 04 utilizaram MotorPosition e AccelStepper para validar movimentos em graus. O microstepping foi consolidado em 1/8. A Fase 02 usou inicialmente uma faixa de -90 a +90 graus; a Fase 03 introduziu a margem operacional de -80 a +80 graus e a regra de perda de referência quando o motor é liberado.

8.2 Ensaio automático de reversão

A Fase 04 e a posterior Fase 06 implementaram ensaio repetitivo do tipo 0 -> +60 -> -60 -> 0. O objetivo era revelar perda de passos, ressonância, limitações de aceleração e assimetrias de reversão. Como não existe encoder no braço, o retorno do software a 0 grau deve ser comparado com uma marca física; caso as marcas não coincidam, a referência foi perdida.

8.3 Fase 06 - valores de trabalho

Parâmetro	Valor na Fase 06	Uso no sistema final
Velocidade padrão	180 graus/s	mantida como velocidade máxima
Aceleração padrão de ensaio	700 graus/s ²	caracterização do atuador
Limite administrativo de teste	720 graus/s e 3000 graus/s ²	apenas proteção contra comandos absurdos
Saturação de controle	-	600 graus/s ² na Fase 09/10

A Fase 06 separa intencionalmente limite administrativo de teste e limite confiável de operação. O controlador final usa 180 graus/s como limite de velocidade e 600 graus/s² como saturação de aceleração, abaixo da aceleração padrão de caracterização de 700 graus/s².

8.4 MotorVelocity: aceleração como entrada

Para as fases de identificação e controle foi criada MotorVelocity. O método commandAcceleration() integra a aceleração comandada no passo de tempo real, obtendo uma referência de velocidade. Essa referência é limitada a +/-180 graus/s, verificada contra os limites angulares e convertida em micropassos/s para setSpeed(). O runSpeed() precisa ser chamado com alta frequência no loop principal.

$$\text{phiDot_ref}(k+1) = \text{sat}[\text{phiDot_ref}(k) + u(k) * dt]$$

Limitação metrológica: O sistema conhece quantos passos foram emitidos, mas não mede diretamente o ângulo mecânico do braço. Perda de passos, folga ou deslizamento não são observáveis pelo software corrente.

PARTE IV

Identificação e modelagem

Obtenção experimental do modelo local em torno da vertical

9. Metodologia de identificação experimental

9.1 Estrutura do modelo

A identificação foi deliberadamente local, isto é, restrita à vizinhança da posição vertical superior. Em vez de depender de todos os parâmetros mecânicos do conjunto, foi escolhida uma estrutura de segunda ordem para o movimento do pêndulo, com aceleração comandada do braço como entrada.

$$\ddot{\beta} = a \beta + b \dot{\beta} + c u$$

9.2 Condições de ensaio

Condição	Valor da Fase 07
Período nominal	4 ms (250 Hz)
Janela de dados	400 ms
Região de topo para preparação	$ \beta \leq 1.5$ grau
Velocidade máxima para topo estável	$ \dot{\beta} \leq 0.25$ rad/s
Estabilidade antes do ensaio	400 ms
Limite de β durante identificação	10 graus
Limite de ϕ durante identificação	10 graus
Velocidade máxima do motor	180 graus/s

9.3 Excitação forçada

O firmware implementa dois perfis de excitação simétricos, G+ e G-. A magnitude é 600 graus/s². Em G+, a aceleração é +600 entre 0 e 60 ms, -600 entre 60 e 180 ms, +600 entre 180 e 240 ms e zero no restante da janela. G- inverte todos os sinais.

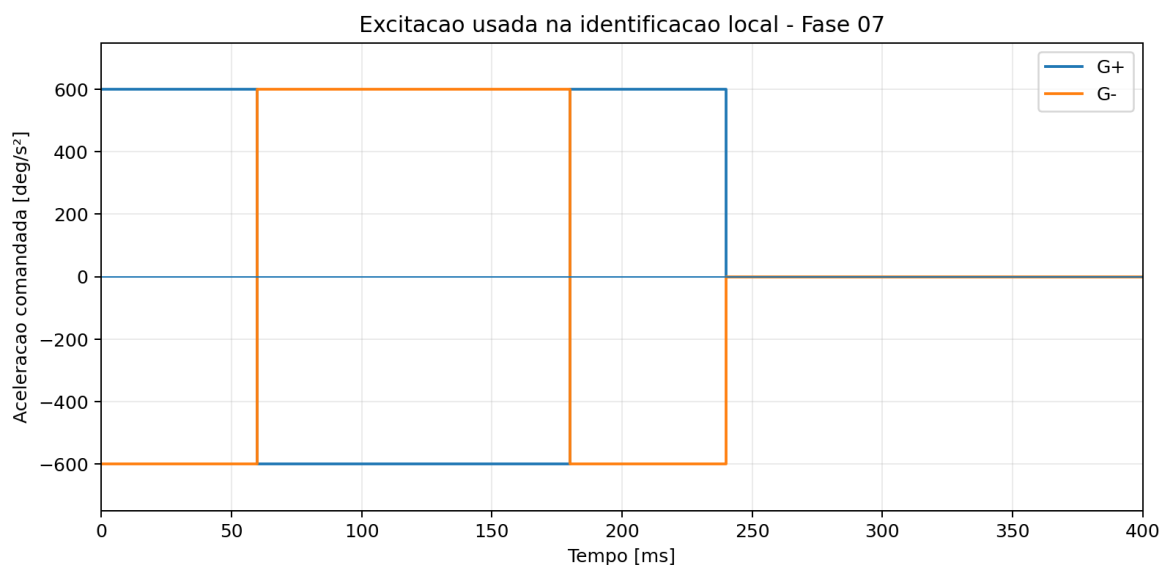


Figura 3 - Perfil de aceleração comandada utilizado na identificação local.

9.4 Preservação do período de amostragem

Um ponto importante da Fase 07 é que nenhuma telemetria é transmitida durante os 400 ms do experimento. Os dados são armazenados em SRAM e impressos somente depois que o motor é parado. Isso evita que a conversão e transmissão serial de números em ponto flutuante perturbem o Ts de 4 ms.

9.5 Uso econômico da SRAM

O Arduino Nano dispõe de apenas 2 kB de SRAM. Por isso cada amostra da Fase 07 é compactada em seis bytes: tempo relativo em uint16_t com unidade de 16 us; beta escalado por 1e5 em int16_t; e betaDot escalado por 1e4 em int16_t. O buffer possui 104 amostras, consumindo 624 bytes. A entrada u não é armazenada; ela é reconstruída posteriormente a partir do tempo e do perfil conhecido.

Unidades da identificação: O firmware de ensaio reconstrói e imprime a excitação em graus/s². Para o modelo consolidado usado no controlador, u é expresso em rad/s²; a conversão para SI é parte da consolidação dos dados.

10. Modelo local identificado

10.1 Resultado consolidado

$$\beta_{ddot} = 21.7 \beta - 3.28 \beta_{Dot} - 0.75 u$$

No modelo acima, beta é expresso em rad, betaDot em rad/s e u em rad/s². O coeficiente positivo 21.7 associado a beta indica a instabilidade local da vertical superior; o termo -3.28 betaDot representa amortecimento efetivo; e o coeficiente -0.75 estabelece o sinal e a intensidade aproximada do acoplamento entre aceleração do braço e aceleração do pêndulo.

10.2 Polos locais sem controle

Desconsiderando a entrada u, a equação característica é $s^2 + 3.28 s - 21.7 = 0$. Os polos são aproximadamente $+3.299 s^{-1}$ e $-6.579 s^{-1}$. A presença de um polo positivo confirma quantitativamente a natureza instável do equilíbrio vertical.

10.3 Natureza do modelo

Este não é um modelo físico global do Furuta. É um modelo experimental local, obtido na região de captura. Ele não descreve o swing-up, grandes ângulos ou saturações do motor. Além disso, u é a aceleração comandada no software, phi não possui encoder independente e betaDot é uma estimativa filtrada; os coeficientes devem, portanto, ser interpretados como parâmetros efetivos da cadeia completa hardware + acionamento + estimador.

PARTE V

Desenvolvimento do controle

Do PD local à realimentação conjunta dos quatro estados

11. Primeiro controlador local: PD do pêndulo

11.1 Lei de controle

$$u = K_p \beta + K_d \dot{\beta}$$

A Fase 08 usa o modelo identificado para projetar um controlador PD em torno do topo. Como o coeficiente de u no modelo é negativo, os ganhos positivos na convenção adotada produzem realimentação estabilizante.

11.2 Escolha dos ganhos

Foram escolhidos polos desejados -4 e -6 para a dinâmica local do pêndulo. Substituindo $u = K_p \beta + K_d \dot{\beta}$ no modelo, a dinâmica fechada fica $\ddot{\beta} = (21.7 - 0.75 K_p) \beta + (-3.28 - 0.75 K_d) \dot{\beta}$. Igualando o polinômio resultante a $(s+4)(s+6) = s^2 + 10s + 24$, obtêm-se os ganhos consolidados:

$$K_p = 60.93 \quad ; \quad K_d = 8.96$$

11.3 Saturação

O comando é limitado a ± 600 graus/s², equivalente a aproximadamente ± 10.472 rad/s². O motor permanece limitado a 180 graus/s.

11.4 Resultado e limitação estrutural

O PD demonstrou capacidade de estabilizar β localmente, mas não regula ϕ . Em um ensaio representativo sem realimentação do braço, o controle terminou por ARM_LIMIT após aproximadamente 1.605 s, com $\phi = +75.375$ graus, enquanto o máximo de $|\beta|$ permaneceu em 4.482 graus. Esse resultado mostrou que estabilizar apenas o pêndulo não garante que o braço permaneça dentro do espaço de trabalho.

12. Captura e entrada no modo BALANCE

12.1 Motivação

Como o swing-up ainda não foi implementado, o pêndulo é levado manualmente até a vizinhança da vertical. O firmware precisa distinguir três situações: aproximação do topo, confirmação de que o pêndulo está suficientemente quieto e detecção da soltura pelo operador. Essa lógica foi formalizada em uma máquina de estados.

Maquina de estados do BALANCE

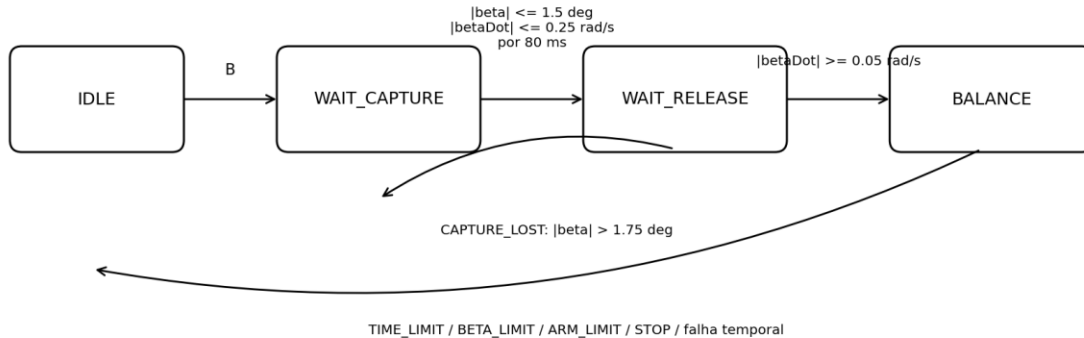


Figura 4 - Máquina de estados usada para captura e BALANCE.

12.2 Critérios atuais

Evento	Critério
CAPTURE_READY	$ beta \leq 1.5$ grau e $ betaDot \leq 0.25$ rad/s durante 80 ms; braço próximo de $\phi = 0$
RELEASE	$ betaDot \geq 0.05$ rad/s
CAPTURE_LOST	$ beta > 1.75$ grau antes da soltura
BETA_LIMIT em BALANCE	$ beta \geq 8$ graus
ARM_LIMIT em BALANCE	$ \phi \geq 75$ graus

A estratégia separa o momento em que o operador está segurando o pêndulo daquele em que o sistema deve assumir o controle. Isso evita iniciar o BALANCE enquanto a mão ainda impõe uma condição artificial ao pêndulo.

13. Da correção independente do braço ao controle conjunto

Após o PD, foi considerada uma correção adicional lenta para conter o braço. Ensaios dessa arquitetura mostraram que simplesmente somar termos independentes de ϕ e ϕDot ao PD pode introduzir dinâmica indesejada. A análise de polos do modelo aumentado confirmou que sinais e ganhos do braço não podem ser escolhidos de forma intuitiva sem considerar o acoplamento com β .

Esse resultado levou à mudança de abordagem: em vez de tratar o pêndulo e o braço como duas correções separadas, os quatro ganhos passaram a ser calculados conjuntamente sobre um único modelo de estados.

Lição de projeto: A realimentação de ϕ não era o problema em si. O problema era adicionar uma malha de braço sem projetar os quatro termos simultaneamente. A Fase 09 resolve exatamente esse ponto.

14. Modelo aumentado de quatro estados

14.1 Hipótese do atuador

A classe MotorVelocity recebe uma aceleração angular comandada e integra essa aceleração para formar a referência de velocidade. Por isso, no modelo de controle, adota-se a aproximação $\phi_{ddot} = u$. Ela descreve bem a estrutura do comando de software, mas não substitui uma medição física de aceleração do braço.

$$\phi_{ddot} = u$$

14.2 Matrizes de estado

Com $x = [\phi, \dot{\phi}, \beta, \dot{\beta}]^T$, o modelo contínuo usado no projeto é:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 21.7 & -3.28 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 1 \\ 0 \\ -0.75 \end{bmatrix}$$

14.3 Controlabilidade

A matriz de controlabilidade $[B, AB, A^2B, A^3B]$ possui posto 4. Portanto, dentro do modelo adotado, os quatro estados são controláveis por uma única entrada u . Esse resultado justifica o posicionamento conjunto dos quatro polos.

14.4 Interpretação do PD no modelo aumentado

Quando $K_{\phi} = K_{\dot{\phi}} = 0$, o controlador regula apenas β e $\dot{\beta}$. Os dois estados do braço permanecem associados a integradores não regulados. Em termos de polos, a dinâmica aumentada contém os polos do PD e dois polos em zero. Essa é a explicação estrutural para a deriva do braço observada experimentalmente.

15. Projeto do controlador de quatro estados

15.1 Polos desejados

$$p = [-0.7, -1.0, -4.0, -6.0]$$

Os polos -4 e -6 preservam a dinâmica rápida escolhida para o pêndulo. Os polos mais lentos -0.7 e -1.0 introduzem regulação do braço sem tornar essa correção tão agressiva quanto a estabilização de β .

15.2 Ganhos

Na convenção MATLAB $u = -Kx$, o vetor calculado possui sinais negativos. No firmware os sinais são incorporados diretamente na soma, resultando em:

$$u = 0.77419355 \phi + 2.31978594 \dot{\phi} + 95.71078856 \beta + 14.31971458 \dot{\beta}$$

Substituindo os ganhos em $A + B F$, com F igual à linha de ganhos usada pelo firmware, os autovalores resultam numericamente em -6, -4, -1 e -0.7, confirmando a implementação da lei planejada.

15.3 Saturação explícita

FourStateController soma os quatro termos em u_{Raw} e limita u a ± 600 graus/s². O campo `saturated` registra a atuação do clamp e alimenta as estatísticas e a telemetria da Fase 10.

PARTE VI

Validação experimental do BALANCE

Ensaaios, desempenho e limites observados

16. Validação experimental do BALANCE

16.1 Ensaios curtos e repetibilidade

A Fase 09 foi testada repetidamente antes do ensaio longo. Quatro execuções representativas encerraram por TIME_LIMIT, três com 5 s e uma com 10 s. Os resultados mostraram que o braço deixou de caminhar sistematicamente até o limite de +/-75 graus, comportamento que ocorria com o PD isolado.

Duração	Max beta [grau]	Max phi [grau]	Max u [grau/s ²]	Saturações
5 s	3.164	31.950	600.0	12
5 s	4.219	34.200	600.0	23
5 s	3.691	39.600	600.0	5
10 s	2.725	32.625	600.0	1

16.2 Ensaio de 100 s bem-sucedido

Um ensaio da Fase 09 permaneceu em BALANCE até o limite de tempo de 100 s. Esse ensaio é a evidência mais forte, até o momento, de estabilização local sustentada no hardware.

Grandeza	Resultado
Motivo do encerramento	TIME_LIMIT
Tempo	100001 ms
Amostras de controle	24998
beta final	-0.615 grau
betaDot final	-0.1233 rad/s
phi final	27.000 graus
phiDot comandada final	-3.441 graus/s
u final	-201.35 graus/s ²
Max beta	1.758 grau
Max betaDot	0.3973 rad/s
Max phi	31.950 graus
Max phiDot	50.146 graus/s
Max u	370.7 graus/s ²
Contagem de saturações	0

O número de amostras é consistente com uma malha nominal de 4 ms: 100 s correspondem a aproximadamente 25 mil ciclos. Nesse teste, a aceleração exigida permaneceu bem abaixo da saturação de 600 graus/s^2 e não houve saturação registrada.

16.3 Ensaio que terminou por perda de equilíbrio

Outro ensaio da mesma estratégia terminou por BETA_LIMIT depois de 25.541 s. Esse resultado é igualmente importante porque mostra que o sucesso de 100 s não implica robustez ilimitada.

Grandeza	Resultado no ensaio BETA_LIMIT
Tempo	25541 ms
Amostras	6383
Max beta	8.086 graus
Max betaDot	2.3021 rad/s
Max phi	43.650 graus
Max phiDot comandada	180.000 graus/s
Max u	600.0 graus/s^2
Saturações	200

Na perda, tanto a velocidade do braço quanto a aceleração de controle atingiram os limites configurados. O termo associado a betaDot chegou a uma contribuição máxima registrada de 1888.8 graus/s^2 antes da saturação do comando total. A interpretação correta é que a dinâmica entrou em uma região na qual o atuador não podia executar a aceleração pedida pelo controlador.

16.4 Conclusão experimental desta etapa

Os ensaios demonstram que a realimentação de quatro estados é funcional e capaz de estabilizar o conjunto por períodos longos, mas ainda não caracterizam probabilisticamente a robustez frente a diferentes condições de captura, atrito, aquecimento, perda de passos, variação do zero do sensor ou distúrbios externos. A Fase 10 foi criada precisamente para registrar a trajetória completa e permitir investigar o que antecede uma eventual perda.

PARTE VII

Integração final e instrumentação

Refatoração da Fase 10, telemetria e operação atual

17. Refatoração e integração final - Fase 10

17.1 Objetivo

A Fase 10 não altera a lei de controle validada. Seu objetivo é separar configuração, estado, controlador, telemetria e coordenação do sistema, reduzindo o main.cpp a uma interface mínima. Os ganhos e limites principais permanecem congelados.

17.2 FurutaConfig.h

Todos os parâmetros operacionais atuais estão concentrados em lib/FurutaSystem/src/FurutaConfig.h. No snapshot analisado, `BALANCE_TEST_TIME_US` está definido como 100000000 us, isto é, 100 s.

Parâmetro	Valor atual
<code>SERIAL_BAUD</code>	250000
<code>CONTROL_PERIOD_US</code>	4000 us
<code>TELEMETRY_PERIOD_US</code>	40000 us
<code>MAX_CONTROL_DT_US</code>	20000 us
<code>BALANCE_TEST_TIME_US</code>	100000000 us (100 s)
<code>MOTOR_MAX_SPEED_DEG_S</code>	180 graus/s
<code>MAX_ACCEL_DEG_S2</code>	600 graus/s ²
<code>ARM_MIN_DEG / ARM_MAX_DEG</code>	-80 / +80 graus
<code>ARM_ABORT_DEG</code>	75 graus
<code>BETA_ABORT_DEG</code>	8 graus

17.3 FurutaSystem

FurutaSystem agrega os objetos de hardware e software, mantém a máquina de estados, adquire x, chama o controlador, envia u ao motor, executa verificações de segurança, processa comandos seriais e produz o resumo de cada ensaio. O método `update()` preserva chamadas frequentes a `motor.update()` e executa o tick de controle quando `nowUs` alcança `nextControlTimeUs`.

Se o loop se atrasar, o código não tenta executar uma rajada de ciclos antigos. `nextControlTimeUs` é reposicionado para o futuro. Durante `BALANCE`, se o dt real ultrapassar 20 ms, o ensaio é encerrado com `CONTROL_OVERRUN`.

17.4 Segurança lógica

- `BETA_LIMIT` encerra o `BALANCE` em $|\text{beta}| \geq 8$ graus.
- `ARM_LIMIT` encerra em $|\text{phi}| \geq 75$ graus.
- `MotorVelocity` também conhece limites absolutos de -80 a +80 graus.
- `MOTOR_COMMAND_REJECTED` encerra o ensaio se a biblioteca do motor rejeitar o comando.
- `STOP` encerra de forma controlada; X executa `emergencyStop` e desabilita o driver.

- Após o BALANCE normal, o motor permanece habilitado e parado; o operador deve usar D antes de reposicionar manualmente o braço.

18. Telemetria do vetor de estado e do controle

18.1 Frequências

A malha de controle continua em 250 Hz. A telemetria é emitida em 25 Hz, uma linha a cada 40 ms. A redução de dez vezes evita transformar a serial em parte dominante do tempo de execução e ainda fornece 2500 pontos em um ensaio de 100 s.

18.2 Formato CSV

```
# TELEMETRY_BEGIN
# t_ms,phi_rad,phiDot_rad_s,beta_rad,betaDot_rad_s,uRaw_rad_s2,u_rad_s2
40,...
80,...
...
# TELEMETRY_END
```

uRaw é a saída matemática da realimentação de estados antes da saturação. u é o valor efetivamente limitado que segue para MotorVelocity. Essa distinção permite identificar exatamente quando o controlador demanda mais aceleração do que o limite de segurança permite.

18.3 Telemetria sem recuperação em rajada

BalanceTelemetry avança nextTimeUs pelo período normal. Se o sistema já estiver atrasado além do próximo instante, o código reposiciona a próxima transmissão para now + periodUs em vez de imprimir várias linhas acumuladas. Essa escolha protege o controle contra uma cascata de mensagens atrasadas.

18.4 Uso esperado dos dados

A telemetria permite reconstruir as curvas $\phi(t)$, $\phi\dot{(t)}$, $\beta(t)$, $\beta\dot{(t)}$, $u_{\text{Raw}}(t)$ e $u(t)$. Com esses dados será possível distinguir deriva lenta do braço, oscilação de baixa frequência, bias de referência, saturação incipiente e eventos que antecedem uma perda de equilíbrio.

19. Procedimento operacional atual

A sequência operacional exibida pelo firmware da Fase 10 é D -> Z -> T -> E -> B. Cada etapa possui uma função específica:

1. D - parar o controle e desabilitar o motor; a referência do braço é marcada como perdida.
2. Posicionar fisicamente o braço no centro de sua faixa de trabalho.
3. Z - definir a posição atual do braço como $\phi = 0$.
4. Com o pêndulo livre, parado e para baixo, executar T para registrar DOWN.
5. E - habilitar o motor.
6. B - armar WAIT_CAPTURE e levar manualmente o pêndulo à vertical.
7. Aguardar CAPTURE_READY; soltar o pêndulo. A detecção de velocidade transfere o sistema para BALANCE.
8. Após o fim do ensaio, usar D antes de mover o braço manualmente.

Segurança operacional: Durante ensaios, manter mãos e objetos fora da faixa de varredura do braço. A movimentação manual deve ser feita com o motor desabilitado. Os limites de software não substituem uma área física livre de obstáculos.

PARTE VIII

Síntese, limitações e continuidade

Estado alcançado e próximos passos de desenvolvimento

20. Resultados consolidados, limitações e riscos técnicos

20.1 Síntese por fase

Fase	Objetivo efetivamente representado no código	Resultado consolidado
00	estrutura e validação do PlatformIO	base de compilação/upload
01	posição e velocidade do pêndulo	primeira validação do sensor; API histórica
02	controle isolado do motor	movimento em graus
03	zero manual e limites	referência do braço e margem operacional
04	limites reais do motor	ensaio repetitivo e ajuste V/A
05	aquisição determinística	250 Hz e betaDot por regressão
06	caracterização do atuador	180 graus/s e ensaios de aceleração/reversão
07	identificação local	dados temporizados e modelo beta_ddot
08	controle PD	estabilização local de beta; braço não regulado
09	controle de quatro estados	regulação conjunta e ensaios longos
10	integração final	refatoração e telemetria do vetor de estado

20.2 Limitações atuais

- Não há encoder independente no braço; ϕ é inferida por passos emitidos.
- ϕ Dot é a referência de velocidade comandada, não uma medição de velocidade mecânica.
- Perda de passos não é detectada automaticamente.
- O modelo de identificação é local e não deve ser extrapolado para grandes ângulos.
- betaDot contém filtragem e atraso efetivo devido à janela de regressão.
- O AS5600 depende da qualidade de alinhamento e do campo do ímã.
- A captura é manual; swing-up automático ainda não faz parte do firmware consolidado.
- Existem ensaios bem-sucedidos longos e também perdas por saturação; a robustez precisa ser quantificada.
- O tempo de ensaio é um limite de teste, não parte da lei de controle.

20.3 Interpretação do estado de maturidade

O projeto já ultrapassou a demonstração de princípio. Existe uma cadeia completa de medição, estimativa, modelo, controle e instrumentação, e o equilíbrio local foi sustentado por 100 s em um ensaio. Entretanto, a ausência de medição independente do braço e a existência de ensaios que atingem saturação impedem classificar o sistema como robusto ou autônomo. O estágio atual é melhor descrito como protótipo experimental funcional de estabilização local.

21. Próximas etapas

A prioridade técnica imediata é utilizar a telemetria da Fase 10 para estudar trajetórias longas e identificar a origem das excursões que eventualmente levam à saturação. Em seguida, as melhorias devem ser incorporadas de forma controlada, preservando a capacidade de comparar cada nova versão com o baseline da Fase 09/10.

1. Executar séries de ensaios de 100 s com telemetria, incluindo capturas a partir dos dois lados da vertical.
2. Analisar $\phi(t)$, $\beta(t)$, velocidades e u_{Raw} antes de eventos de saturação.
3. Verificar fisicamente a repetibilidade da referência do braço e a ocorrência de perda de passos.
4. Avaliar a inclusão de encoder no braço; isso converteria ϕ e $\dot{\phi}$ em estados medidos/estimados a partir de medição física.
5. Reavaliar o modelo local com a nova instrumentação e estimar incertezas dos coeficientes.
6. Estudar observador de estados ou filtros adicionais apenas depois de quantificar o benefício esperado.
7. Implementar swing-up como modo separado e projetar a transição SWING_UP -> CAPTURE -> BALANCE.
8. Executar ensaios de robustez, aquecimento, perturbação e repetibilidade antes de ampliar limites operacionais.

22. Conclusão

O projeto FURUTA V0 foi desenvolvido de maneira incremental e experimentalmente orientada. A cadeia de aquisição do pêndulo evoluiu de leitura angular básica para uma estimativa de velocidade baseada em regressão; o acionamento evoluiu de posição para aceleração comandada; a planta foi identificada localmente; o PD demonstrou a necessidade de controlar também o braço; e o modelo aumentado permitiu projetar uma realimentação conjunta dos quatro estados.

A Fase 10 consolidou a arquitetura sem alterar os ganhos validados: o main.cpp tornou-se mínimo, parâmetros passaram a ficar centralizados, a lei $u = Kx$ foi encapsulada e a telemetria passou a registrar o vetor de estado e o comando antes e depois da saturação. O resultado atual é uma plataforma apropriada para a próxima etapa científica do trabalho: transformar observações de ensaios longos em evidências quantitativas sobre robustez, bias, saturação e perda de referência, antes de avançar para swing-up e autonomia completa.

PARTE APÊNDICES

Dados de referência e auditoria

Informações operacionais e de rastreabilidade do snapshot

Apêndice A - Pinagem e parâmetros principais

Item	Valor
STEP	D6
DIR	D8
ENABLE	D4
Motor	NEMA 17, 200 passos/rev
Microstepping	1/8
Driver	A4988
Sensor	AS5600
Velocidade máxima do braço	180 graus/s
Aceleração máxima de controle	600 graus/s ²
Limite absoluto do braço	-80 a +80 graus
Abort operacional do braço	+/-75 graus
Abort do pêndulo	+/-8 graus
Controle	250 Hz
Telemetria	25 Hz
Serial final	250000 baud
Tempo de ensaio atual	100 s

Apêndice B - Mapa das fases

Env PlatformIO	Diretório fonte	Função real no snapshot
fase_00_estrutura	src/fase_00_estrutura	estrutura inicial
fase_01_sensor	src/fase_01_sensor	sensor - versão histórica
fase_02_motor	src/fase_02_motor	motor isolado
fase_03_referencia_limites	src/fase_03_referencia_limites	zero e limites do braço
fase_04_limites_motor	src/fase_04_limites_motor	ensaios de limites do motor
fase_05_aquisicao	src/fase_05_aquisicao	aquisição a 250 Hz

Env PlatformIO	Diretório fonte	Função real no snapshot
fase_06_atuador	src/fase_06_atuador	caracterização do atuador
fase_07_identifica	src/fase_07_identifica	identificação local
fase_08_projetoPD	src/fase_08_projetoPD	controle PD
fase_09_projetoMotorPendulo	src/fase_09_projetoMotorPendulo	controle de quatro estados
fase_10_integracao_final	src/fase_10_integracao_final	integração final

Apêndice C - Comandos seriais da implementação atual

Comando	Função
T	calibrar o pêndulo para baixo (DOWN)
Z	definir a posição atual do braço como $\phi = 0$
E	habilitar o motor
D	parar/desabilitar o motor e invalidar a referência do braço
B	armar captura e BALANCE
STOP	parar o controle
X	emergency stop
S	status
H	ajuda

Durante BALANCE, o parser aceita STOP e X como comandos efetivos e evita processar operações que poderiam gerar longa impressão serial e perturbar a temporização da malha.

Apêndice D - Estrutura de diretórios

```

FURUTA_V0/
|-- platformio.ini
|-- README.md
|-- lib/
|   |-- BalanceTelemetry/src/
|   |-- FourStateController/src/
|   |-- FurutaSystem/src/
|   |-- MotorPosition/src/
|   |-- MotoreVelocity/src/
|   |-- PendulumPosition/src/
|   |-- PendulumVelocity/src/
|   |-- StateVector/src/
|-- src/
|   |-- fase_00_estrutura/
|   |-- fase_01_sensor/
|   |-- fase_02_motor/
|   |-- fase_03_referencia_limites/
|   |-- fase_04_limites_motor/
|   |-- fase_05_aquisicao/
|   |-- fase_06_atuador/
|   |-- fase_07_identifica/
|   |-- fase_08_projetoPD/
|   |-- fase_09_projetoMotorPendulo/
|   |-- fase_10_integracao_final/

```

Apêndice E - Auditoria de consistência do repositório

A análise do ZIP identificou alguns pontos de organização que não alteram a descrição funcional da Fase 10, mas devem ser conhecidos para evitar que documentação histórica seja confundida com configuração atual.

Observação	Impacto / recomendação
README.md descreve estado apenas até a Fase 04 e ainda cita 115200 baud como configuração geral.	Tratar o README como histórico; atualizar para refletir Fases 05-10 e 250000 baud na implementação final.
Comentários do platformio.ini chamam Fase 07 de swing-up e Fase 08 de captura, mas os fontes reais são identificação local e controle PD.	Corrigir os comentários do .ini; a documentação usa a função real encontrada nos main.cpp.
src/fase_01_sensor/main.cpp usa uma API antiga de PendulumPosition/PendulumVelocity diferente da biblioteca corrente.	As fases antigas são snapshots históricos; para builds reprodutíveis de todas as fases, versionar também as bibliotecas por tag/commit.
A pasta lib/MotoreVelocity possui grafia diferente da classe/cabeçalho MotorVelocity.	Renomear futuramente para MotorVelocity por consistência; testar o build após a alteração.
Existe lib/FurutaSystem/FurutaSystem.h fora de src, contendo conteúdo antigo de configuração, além do FurutaSystem.h real em lib/FurutaSystem/src/.	Remover ou arquivar o arquivo residual após confirmar que não é usado pelo build.
platformio.ini declara dependências sem versão fixa, embora o cache contenha AS5600 0.6.7 e AccelStepper 1.64.	Fixar versões quando o objetivo passar de desenvolvimento para reprodução/entrega.
Alguns comentários da Fase 09 ainda dizem "primeiro ensaio 5 s" enquanto a constante corrente está em 100 s.	Usar a constante como fonte de verdade e evitar textos hardcoded de duração. A Fase 10 já imprime a duração a partir da configuração.

Estado da auditoria: O usuário informou que a Fase 10 refatorada funcionou no hardware. O ambiente desta documentação não possui PlatformIO instalado; por isso não foi executada uma recompilação independente do ZIP. A validação documental foi feita por inspeção do código e pelos resultados de execução fornecidos.

Referências de implementação

1. Rob Tillaart. Biblioteca Arduino AS5600. Versão 0.6.7 encontrada no cache PlatformIO do snapshot.
2. Mike McCauley / Patrick Wasp. AccelStepper. Versão 1.64 encontrada no cache PlatformIO do snapshot.
3. PlatformIO. Configuração do projeto em platformio.ini para atmelavr / nanoatmega328 / Arduino.
4. Código-fonte do próprio projeto FURUTA V0, diretórios src/ e lib/, snapshot analisado em 09/08/2026.